Heron: Phishing URL Detection - Code

Author: Pranitha U P | IBM SkillsBuild Cyber Security BootCamp 2026 (BharatCares)
File: Pranitha_U_P_Heron.ipynb (Jupyter Notebook, source exported to PDF)

Heron: Phishing URL Detection

Author: Pranitha U P
Program: IBM SkillsBuild Cyber Security BootCamp 2026 (BharatCares)
Heron classifies a URL as phishing (malicious) or legitimate using machine learning on lexical features extracted from the URL string itself. No network request is made to the suspicious site, so the approach is fast and safe.
Pipeline: load data, clean, extract features, explore, train models, evaluate, save the best model, predict on new URLs.

1. Setup

```
import re, warnings
from urllib.parse import urlparse
from collections import Counter
import math

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import joblib

from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             roc_auc_score, confusion_matrix, classification_report, RocCurveDisplay)

warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
RANDOM_STATE = 42
```

2. Load the dataset

Set DATA_PATH to the dataset CSV (see README for the download link). Accepted formats:
* Kaggle *Phishing Site URLs*: columns URL, Label (bad / good)
* Any CSV with a URL column and a label column (1/0, phishing/legitimate, bad/good)
If the file is not found, a small demo dataset is generated so the notebook still runs end to end. Results on the demo data are for illustration only; use the real dataset for final numbers.

```
import os
DATA_PATH = "phishing_site_urls.csv"   # <-- change if your file is elsewhere
URL_COL, LABEL_COL = "URL", "Label"

def make_demo_data(n=1500, seed=RANDOM_STATE):
    rng = np.random.default_rng(seed)
    good_domains = ["google.com","wikipedia.org","github.com","python.org","ibm.com","amazon.in",
                    "flipkart.com","vtu.ac.in","nptel.ac.in","microsoft.com","bbc.com","stackoverflow.com
    "]
    paths = ["", "about", "docs/index.html", "products/list", "news/2026/10", "learn/python", "search?q=d
    ata"]
    bad_words = ["login","verify","secure","update","account","bank","confirm","signin","wallet","free-gi
    ft"]
    tlds = ["xyz","top","tk","ml","info","click","ru","cn"]
    rows = []
    for _ in range(n//2):
        d = rng.choice(good_domains); p = rng.choice(paths)
        slug = "".join(rng.choice(list("abcdefghijklmnopqrstuvwxyz"), rng.integers(3, 12)))
        extra = rng.choice(["", f"/{slug}", f"/{slug}/{rng.integers(1, 9999)}", f"?ref={slug}"])
        scheme = rng.choice(["https", "https", "http"])
        rows.append((f"{scheme}://www.{d}/{p}{extra}".replace("/?", "?"), "good"))
    for _ in range(n//2):
        brand = rng.choice(["paypal","sbi","hdfc","amazon","netflix","apple","icici","microsoft"])
```

```
        w1, w2 = rng.choice(bad_words, 2)
        tld = rng.choice(tlds)
        style = rng.integers(0, 3)
        if style == 0:
            u = f"http://{brand}-{w1}.{w2}-{rng.integers(100,999)}.{tld}/{w1}/index.php?id={rng.integers(
    10000,99999)}"
        elif style == 1:
            u = f"http://{rng.integers(1,255)}.{rng.integers(1,255)}.{rng.integers(1,255)}.{rng.integers(
    1,255)}/{brand}/{w1}.html"
        else:
            u = f"http://{brand}.com.{w1}-{w2}.{tld}/@/{w1}?token={rng.integers(10**6,10**7)}"
        rows.append((u, "bad"))
    return pd.DataFrame(rows, columns=[URL_COL, LABEL_COL]).sample(frac=1, random_state=seed).reset_index
    (drop=True)

USING_DEMO = not os.path.exists(DATA_PATH)
df = make_demo_data() if USING_DEMO else pd.read_csv(DATA_PATH)
print("Using DEMO data" if USING_DEMO else f"Loaded {DATA_PATH}")
print(df.shape)
df.head()
```

3. Clean the data

```
def normalise_label(v):
    s = str(v).strip().lower()
    if s in {"1","bad","phishing","malicious","malware","defacement","yes","true"}: return 1
    if s in {"0","good","legitimate","benign","no","false"}: return 0
    if s == "-1": return 1   # UCI phishing-websites convention: -1 = phishing
    return np.nan

df = df[[URL_COL, LABEL_COL]].copy()
df[URL_COL] = df[URL_COL].astype(str).str.strip()
df["label"] = df[LABEL_COL].map(normalise_label)
before = len(df)
df = df.dropna(subset=["label"]).drop_duplicates(subset=[URL_COL]).reset_index(drop=True)
df["label"] = df["label"].astype(int)
print(f"Rows: {before} -> {len(df)} after removing unknown labels and duplicate URLs")
print(df["label"].value_counts().rename({0:"legitimate",1:"phishing"}))
```

4. Feature engineering

Features come only from the URL text:

Group	Examples
Length	URL, hostname, path, query length
Characters	counts of . - _ / @ ? = % digits, letters, special characters
Host	uses IP address, number of subdomains, TLD, https, www
Suspicious cues	@ symbol, // in path, URL shortener, suspicious keywords (login, verify, secure...)
Statistical	Shannon entropy of the URL and host

```
SUSPICIOUS_WORDS = ["login","signin","verify","secure","update","account","bank","confirm","password",
                    "wallet","free","bonus","gift","webscr","paypal","ebay","support","billing","alert","
    suspend"]
SHORTENERS = ["bit.ly","goo.gl","tinyurl.com","t.co","ow.ly","is.gd","buff.ly","rebrand.ly","cutt.ly"]
RISKY_TLDS = {"xyz","top","tk","ml","ga","cf","gq","click","link","info","ru","cn","work","zip","country"
    }
IP_RE = re.compile(r"^(?:\d{1,3}\.){3}\d{1,3}$")

def shannon_entropy(s):
    if not s: return 0.0
    counts = Counter(s); n = len(s)
    return -sum((c/n) * math.log2(c/n) for c in counts.values())

def extract_features(url):
    url = str(url).strip()
    parsed = urlparse(url if "://" in url else "http://" + url)
    host = (parsed.hostname or "").lower()
    path, query = parsed.path or "", parsed.query or ""
    parts = host.split(".") if host else []
    tld = parts[-1] if len(parts) > 1 else ""
    letters = sum(c.isalpha() for c in url); digits = sum(c.isdigit() for c in url)
    f = {
        "url_len": len(url), "host_len": len(host), "path_len": len(path), "query_len": len(query),
```

```
        "dots": url.count("."), "hyphens": url.count("-"), "underscores": url.count("_"),
        "slashes": url.count("/"), "at_symbol": int("@" in url), "question": url.count("?"),
        "equals": url.count("="), "ampersand": url.count("&"), "percent": url.count("%"),
        "digits": digits, "letters": letters, "digit_ratio": digits / max(len(url), 1),
        "special_chars": len(url) - letters - digits,
        "host_digits": sum(c.isdigit() for c in host), "host_hyphens": host.count("-"),
        "is_ip": int(bool(IP_RE.match(host))),
        "num_subdomains": max(len(parts) - 2, 0),
        "https": int(parsed.scheme == "https"),
        "has_www": int(host.startswith("www.")),
        "risky_tld": int(tld in RISKY_TLDS),
        "double_slash_in_path": int("//" in path),
        "shortener": int(any(s in host for s in SHORTENERS)),
        "suspicious_words": sum(w in url.lower() for w in SUSPICIOUS_WORDS),
        "path_depth": path.count("/"),
        "has_port": int(parsed.port is not None) if _safe_port(parsed) else 0,
        "url_entropy": shannon_entropy(url), "host_entropy": shannon_entropy(host),
    }
    return f

def _safe_port(p):
    try: p.port; return True
    except ValueError: return False

X = pd.DataFrame([extract_features(u) for u in df[URL_COL]])
y = df["label"].values
print(X.shape)
X.head()
```

5. Exploratory analysis

```
fig, ax = plt.subplots(1, 3, figsize=(15, 4))
sns.countplot(x=df["label"].map({0:"Legitimate",1:"Phishing"}), ax=ax[0]); ax[0].set_title("Class balance
    ")
sns.boxplot(x=df["label"].map({0:"Legitimate",1:"Phishing"}), y=X["url_len"], ax=ax[1], showfliers=False)
    ; ax[1].set_title("URL length")
sns.boxplot(x=df["label"].map({0:"Legitimate",1:"Phishing"}), y=X["suspicious_words"], ax=ax[2], showflie
    rs=False); ax[2].set_title("Suspicious keywords")
plt.tight_layout(); plt.show()

corr = X.assign(label=y).corr()["label"].drop("label").sort_values()
corr.plot(kind="barh", figsize=(7, 9), title="Correlation of each feature with the phishing label")
plt.show()
```

6. Train / test split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=RANDOM_STATE)
print("Train:", X_train.shape, " Test:", X_test.shape)
```

7. Train and compare models

```
models = {
    "Logistic Regression": make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000, class_weight
    ="balanced")),
    "Random Forest": RandomForestClassifier(n_estimators=300, n_jobs=-1, class_weight="balanced", random_
    state=RANDOM_STATE),
    "Gradient Boosting": GradientBoostingClassifier(random_state=RANDOM_STATE),
}
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
results, fitted = [], {}
for name, model in models.items():
    cv_f1 = cross_val_score(model, X_train, y_train, cv=cv, scoring="f1").mean()
    model.fit(X_train, y_train); fitted[name] = model
    pred = model.predict(X_test); proba = model.predict_proba(X_test)[:, 1]
    results.append({"Model": name, "CV F1": cv_f1,
        "Accuracy": accuracy_score(y_test, pred), "Precision": precision_score(y_test, pred),
        "Recall": recall_score(y_test, pred), "F1": f1_score(y_test, pred),
        "ROC-AUC": roc_auc_score(y_test, proba)})
results_df = pd.DataFrame(results).set_index("Model").round(4)
results_df
```

8. Evaluate the best model

```
best_name = results_df["F1"].idxmax()
best = fitted[best_name]
print("Best model:", best_name)
pred = best.predict(X_test)
print(classification_report(y_test, pred, target_names=["Legitimate", "Phishing"]))

fig, ax = plt.subplots(1, 2, figsize=(11, 4))
sns.heatmap(confusion_matrix(y_test, pred), annot=True, fmt="d", cmap="Blues", ax=ax[0],
            xticklabels=["Legit", "Phish"], yticklabels=["Legit", "Phish"])
ax[0].set_title("Confusion matrix"); ax[0].set_xlabel("Predicted"); ax[0].set_ylabel("Actual")
for name, m in fitted.items():
    RocCurveDisplay.from_estimator(m, X_test, y_test, ax=ax[1], name=name)
ax[1].set_title("ROC curves"); plt.tight_layout(); plt.show()

if hasattr(best, "feature_importances_"):
    imp = pd.Series(best.feature_importances_, index=X.columns).sort_values(ascending=False).head(15)
    imp.sort_values().plot(kind="barh", figsize=(7, 6), title=f"Top 15 features ({best_name})")
    plt.show()
```

9. Save the model and predict on new URLs

```
joblib.dump({"model": best, "features": list(X.columns), "name": best_name}, "heron_model.joblib")

def heron_predict(url, threshold=0.5):
    bundle = joblib.load("heron_model.joblib")
    row = pd.DataFrame([extract_features(url)])[bundle["features"]]
    p = bundle["model"].predict_proba(row)[0, 1]
    return {"url": url, "phishing_probability": round(float(p), 3),
            "verdict": "PHISHING" if p >= threshold else "LEGITIMATE"}

tests = [
    "https://www.wikipedia.org/",
    "https://github.com/python/cpython",
    "http://paypal-verify.secure-login-482.xyz/login/index.php?id=88213",
    "http://192.168.44.12/sbi/confirm.html",
    "http://amazon.com.account-update.top/@/signin?token=5523110",
]
pd.DataFrame([heron_predict(u) for u in tests])
```

10. Conclusion and limitations

* Lexical URL features alone give a strong, fast baseline for phishing detection.
* Limitations: no page content, WHOIS/domain age, or reputation lookups; attackers can craft URLs that look benign; the dataset may not reflect the newest phishing campaigns.
* Future work: add domain-age and DNS features, character-level deep learning models (CNN/LSTM) on raw URLs, a browser extension or Flask/Streamlit front end, and periodic retraining.

> Note: this copy was run on the built-in demo data. Re-run with the real dataset and update the report with final metrics.